**UNIVERSIDAD SAN FRANCISCO DE QUITO**
**COLEGIO: CIENCIAS E INGENIERÍAS**
**Semestre:** 202520 - Segundo Semestre 2025/2026

# Phase 1 Project Specification

*Simulated Bitcoin Exchange*

## How Phase 1 will be graded

This phase will be graded by running your published Docker images directly from Docker Hub. Your code will not be rebuilt locally during grading. All published images must run on both ARM and AMD CPUs.

You must still upload the full source code to D2L. Include a README.txt file with the submission. The README.txt must list all team members exactly as they appear in D2L, describe each person's contribution, and provide complete instructions for running this phase of the project.

Each phase builds on the previous one. The project consists of three phases, and the third phase is the final project.

## 1. Big picture

The system will use the Docker Hub image **bitcoin/bitcoin:29.3** to run two full Bitcoin Core nodes in a private regtest network. Each node will run in its own Docker container on the same machine. Because both containers run the same Bitcoin software, either container may be used for standard Bitcoin Core operations such as wallet creation and address management.

At the start of the simulation, no Bitcoin exists. Bitcoin is introduced only through mining. The system will include ten additional Docker containers, each running a Python miner process. The system will also include a control layer that manages a user-configured spendable Bitcoin threshold. In Phase 1, the user will set this threshold through a React interface.

The configured threshold will be sent to a Node.js server through a React interface, which will relay it to the control API inside each Docker miner container. Each miner container will then apply that threshold to the Python miner process running locally in the same container.

Each miner must report its current mining results to the Node.js server. The Node.js server must push those updates to a React table displayed in the user's browser. Each row in the table must show:

- miner number
- Bitcoin gained so far
- Bitcoin available for spending after maturation

The miners must compete to create the next block by constructing blocks that satisfy Bitcoin's block construction rules and proof-of-work requirements.

Once a miner has accumulated enough mature Bitcoin to meet the user-configured threshold, that amount must be transferred to the exchange deposit address for that miner. The exchange must then create an account for that miner. At the beginning of the simulation, miner accounts will hold only Bitcoin. In addition, the exchange must contain 30 client accounts that initially hold only U.S. dollars, with a starting balance of $30,000 USD in each account.

All exchange client accounts must be viewable. You may choose the interface for viewing them using the React library, but the README.txt must clearly explain how a user can access any account.

## 2. Two systems in this project

| Rule | Meaning |
|---|---|
| Blockchain system | Handles mining, wallets, Bitcoin addresses, blocks, confirmations, and on-chain BTC transactions. |
| Exchange system | Handles internal client accounts, internal BTC balances, internal USD balances, and the records the exchange uses for later trading. |
| Important boundary | BTC is not considered to be inside the exchange simply because a miner wallet holds BTC. Bitcoin enters the exchange only after an on-chain transaction sends it to the exchange address and the backend verifies receipt. |

## 3. Fixed technology choices

| Part | Required choice | Why |
|---|---|---|
| Frontend | React | This is the library used for the graphical interface. |
| Backend | Express / Node.js | This is the main control layer. It stores the threshold and coordinates the system. |

| | | |
|---|---|---|
| Miner service | Python | Each miner runs Python code plus a small REST API. |
| Miner API | FastAPI or Flask | Use a simple REST API inside each miner container. |
| Bitcoin nodes | bitcoin/bitcoin in regtest | This provides a private Bitcoin test network. |
| Database | SQLite | This keeps Phase 1 simple. |
| Orchestration | Docker Compose | The user must be able to start the system with Compose. |
| Publishing | Docker Hub multi-architecture images | Your images must work on amd64 and arm64. |

## 4. Mining simulation and payout model

The project must simulate a realistic Bitcoin mining workflow on a private regtest network, not on mainnet. Ten Python miner processes will run in ten separate Docker containers. Each miner container must also include a small server that communicates with a central coordinator implemented as the Node.js main server. The coordinator is responsible for assigning each miner a target payout amount, tracking how much each miner has earned from valid blocks, and determining when mined rewards become spendable.

In Bitcoin, the block reward is created in the block's coinbase transaction. That reward cannot be spent until it has received 100 additional block confirmations. This rule is known as coinbase maturity. For that reason, the system must distinguish between immature rewards and mature rewards, and only mature rewards may be transferred.

Each miner must communicate with an assigned Bitcoin Core regtest node through RPC. The miner must request work with **getblocktemplate**, construct a fully valid candidate block, include a valid coinbase transaction and the non-coinbase transactions provided by the node's mempool template, compute the merkle root, assemble the block header, and perform its own proof-of-work search by varying the nonce and extranonce values. Any discovered block must be submitted back to the assigned node with **submitblock**.

All miners are considered to be competing on the same regtest network through different nodes. Therefore, when one miner finds and submits a valid block, the other miners must detect that their current template is stale and must request a new template before continuing.

Once a miner's mature balance reaches its assigned threshold, the simulation must transfer that threshold amount to the exchange’s  deposit address, and credited to miner's exchange account. Any additional mature Bitcoin beyond that threshold must be transferred to a separate system treasury address.

The system must not transfer rewards immediately when a block is found. Instead, it must wait until the reward has matured, then determine how much of the mature balance satisfies the miner's required threshold and how much remains as surplus. The purpose of this design is to model the real separation between block discovery, reward maturity, and spendable payout while keeping the project manageable for a course assignment.

## 5. Be aware of updates to this specification

Because class time will be devoted to this project, and because students may raise valid questions and concerns, the specification may undergo minor revisions expanded with additional details.